

Research Plan

Neuro-Symbolic Verification for LLM-Based Smart Home Temporal Scheduling Agents

*Combining SMT Constraint Solving with Small Language Models
to Fix the Deferred-Feedback Problem in IoT Workflow Scheduling*

Soumava

MTech Research — September 2026

Target Venue: IEEE Internet of Things Journal — Special Issue on "AI Agent Enabled Small-Large Model Collaboration for IoT" (Deadline: Dec 15, 2026)

Note: *This entire project is simulation-based. No physical hardware is required. All experiments run inside the SimuHome simulator on a single GPU (Colab Pro / Kaggle).*

Part 1: The Papers You Are Working With

This section covers the four papers that form the foundation of your research. Each is explained in plain language with analogies, so you understand what it does and why it matters before reading it in full.

1.1 SimuHome (ICLR 2026 Oral)

Full Title: SimuHome: A Temporal- and Environment-Aware Benchmark for Smart Home LLM Agents
Authors: Gyuhyeon Seo, Jungwoo Yang, Junseong Pyo, Nalim Kim, Jonggeun Lee, Yohan Jo (Seoul National University / Hanyang University)
Published: ICLR 2026 (Oral Presentation) — arXiv: 2509.24282
Code: github.com/holi-lab/SimuHome

What Problem Does It Solve?

Before SimuHome, smart home benchmarks were like board games played on a frozen board: you told a device to turn on, the benchmark checked "did it turn on?" and that was it. Nothing happened to the room. Temperature didn't drop when you turned on the AC, humidity didn't change when you ran the humidifier, and time didn't pass between actions. This meant there was no way to test whether an AI agent could handle real scheduling problems like "turn on the kitchen light when the dishwasher finishes" or "make sure both the washer and dryer are done before 6 PM."

Analogy: Imagine testing a pilot in a flight simulator where the altitude gauge never moves, fuel never runs out, and weather never changes. You could test whether they know which button to press, but you could never test whether they can actually fly. SimuHome is the first flight simulator for smart home AI that actually simulates physics.

What Did They Build?

SimuHome consists of two parts:

A. The Simulator. A virtual smart home built on the Matter protocol (the actual industry standard that real smart devices like Google Home and Apple HomeKit speak). It models 17 device types (AC, dishwasher, washer, humidifier, lights, heat pump, TV, robot vacuum, etc.). Four environmental variables — temperature, illuminance, humidity, and air quality — respond continuously to device actions through an "Aggregator" module. Turn on the AC, and the room temperature drops gradually over simulated minutes, not instantly. The simulator runs in "ticks" (0.1 real-world seconds each) and can accelerate time so you don't have to wait 40 real minutes for a scheduled action to execute. The agent interacts through 13 API tools: `get_room_devices`, `execute_command`, `schedule_workflow`, `get_current_time`, `get_attribute`, `get_device_structure`, and others.

B. The Benchmark. 600 hand-validated episodes across 12 query types (6 types, each with a feasible and an infeasible/trap version):

Q T	Name	Example Query	What Agent Must Do
Q T1	Environment Perception	"How humid is the kitchen?"	Read sensor values correctly

Q T	Name	Example Query	What Agent Must Do
Q T2	Implicit Intent	"It feels stuffy in here"	Infer user wants humidity control
Q T3	Explicit Control	"Set AC fan to 50%"	Execute exact device commands
Q T4 -1	Future Scheduling	"Turn off lights in 10 min"	Calculate absolute time, schedule
Q T4 -2	Dependency Scheduling	"When dishwasher finishes, turn off lights"	Read device countdown, chain actions
Q T4 -3	Concurrent Scheduling	"Both washer and dryer done by 6 PM"	Coordinate two device timelines

Table 1: SimuHome's 6 query types (each has a feasible + infeasible variant = 12 total).

Feasible episodes are scored automatically by comparing the simulator's final state to a pre-defined goal state. Infeasible episodes (where the request is impossible due to contradictory timing, non-existent devices, or physical limits) are scored by an LLM judge that checks whether the agent correctly refused and explained why.

Key Results

Model	QT1-F	QT2-F	QT3-F	QT4-1-F	QT4-2-F	QT4-3-F
GPT-4.1 (best)	98%	44%	84%	50%	46%	34%
Gemini-2.5-Flash	92%	66%	82%	22%	40%	12%
Qwen3-235B	86%	32%	84%	26%	38%	28%
Llama-4-Maverick	96%	52%	88%	22%	18%	32%

Table 2: Success rates (%) on feasible episodes. QT4 (scheduling) is the clear bottleneck.

The pattern is stark: simple perception (QT1) and explicit commands (QT3) are largely solved at 84-98%. Temporal scheduling (QT4) is where everything falls apart. The best model (GPT-4.1) peaks at 50% on QT4-1 and drops to 34% on QT4-3. On infeasible QT4 episodes, Contradiction Blindness (the model doesn't even notice the timing is impossible) accounts for 75-90% of all errors.

KEY FINDING — The Deferred Feedback Problem (Section 6.2 of the paper): When agents call `schedule_workflow`, the tool returns only a bare "scheduled successfully" acknowledgment. It does NOT check whether the scheduled steps will actually be executable when their time comes. Compare this with QT3 (explicit commands), where 40% of successes involved recovery after an initial tool-call error — the error message gave the agent immediate feedback to self-correct. Scheduling gives no such signal. The agent commits blind and only finds out it was wrong after time has passed in the simulator.

Analogy: Imagine booking a flight, a hotel, and a rental car for a trip. With QT3-style tasks, each booking immediately tells you "sorry, that's not available" and you rebook. With QT4-style scheduling, imagine a system that accepts ALL your bookings without checking anything, and you only discover the hotel was full when you arrive at midnight. That's the deferred feedback problem.

1.2 AGENT-C (arXiv, December 2025)

Full Title: Enforcing Temporal Constraints for LLM Agents

arXiv: 2512.23738

Key Idea: Use SMT (Satisfiability Modulo Theories) solvers to enforce temporal constraints on LLM agent tool calls via constrained generation.

What Problem Does It Solve?

LLM agents that call tools (APIs, databases, web services) can violate temporal rules: calling things out of order, skipping required steps, or doing something that invalidates a prior action. AGENT-C introduces a domain-specific language (DSL) where you write temporal rules like "never call API-B before API-A has returned" or "always confirm with the user before deleting." These rules get compiled into first-order logic, and a Z3 SMT solver checks every tool call the LLM tries to make, blocking any that would violate the rules. This happens during generation (constrained decoding), not after.

What Architecture Did They Develop?

The architecture has three layers: (1) a DSL for writing temporal constraints as rules over tool-call sequences, (2) a compiler that translates these rules into first-order logic formulas, and (3) a Z3 SMT solver that evaluates satisfiability at each decoding step, blocking the LLM from emitting any tool call that would make the constraint set unsatisfiable. The LLM never sees the solver — it simply finds that certain tokens are masked out during generation.

Analogy: Think of a spell-checker that works as you type, but instead of checking spelling, it checks logical consistency. If you start typing a sentence that would contradict what you wrote three paragraphs ago, the keyboard refuses to let you finish that word. AGENT-C is that keyboard, but for AI tool calls.

Why It Matters For Your Work

AGENT-C proves the core mechanism works: SMT solvers can enforce temporal constraints on LLM tool calls in real time. But AGENT-C operates in a world of **discrete, boolean events** — "did tool A run before tool B?" It does NOT handle: (a) continuous environmental variables (temperature dropping at 0.5 degrees per minute), (b) device-specific durations (a dishwasher takes 59 minutes, not an instant), or (c) multi-device coupled dynamics (running the AC and the heater simultaneously in the same room). This is the gap your work fills. You extend SMT-verified temporal constraints from boolean event ordering to physics-aware, duration-aware smart home scheduling.

CRITICAL CHECK: You must confirm this gap exists by reading AGENT-C sections 3-4 in full. If their constraint language already supports real-valued durations and continuous state, your novelty claim shrinks significantly. From the abstract and related-work section, this does not appear to be the case — but verify before committing.

1.3 EvoPlan (arXiv, July 2026)

Full Title: EvoPlan: Evolutionary Neuro-Symbolic Robot Planning with Spatio-Temporal Guarantees

arXiv: 2607.06724

Key Idea: LLM proposes plans in PDDL, symbolic validators accept/reject, surviving plans evolve over iterations.

What Problem Does It Solve?

Robot planning needs both flexibility (understanding "clean the table but don't touch the diary") and safety guarantees (never exceed speed limits, maintain safe distance). LLMs are good at the first but terrible at the second. Pure symbolic planners handle safety but can't parse natural language. EvoPlan combines both: an LLM (Qwen3-32B, running locally) proposes plans written in PDDL (a formal planning language), and a cascade of symbolic validators — including Signal Temporal Logic (STL) constraints mined from demonstration data — filter out unsafe plans. Surviving plans "evolve" over iterations, improving through the LLM's ability to repair plans given validator feedback.

Analogy: Imagine a team of architects (the LLM) brainstorming building designs, while a structural engineer (the symbolic validator) checks each design against building codes. Designs that pass get refined further; designs that fail get sent back with specific notes on what's wrong. After several rounds, you get a building that's both creative and structurally sound. EvoPlan is this process for robot actions.

Why It Matters For Your Work

EvoPlan gives you the **architectural template** for your system: the propose-validate-repair loop. Your system will follow the same pattern — a neural model proposes, a symbolic checker validates, and feedback flows back for revision — but in the smart home / IoT domain instead of robotics. The key difference: EvoPlan's constraints are about spatial movement and collision avoidance; yours are about temporal scheduling, device durations, and environmental physics. The loop structure transfers; the constraint formulation is new.

1.4 Planning via Neuro-Symbolic Reasoning (Jha et al., 2023)

Full Title: Planning using Neuro Symbolic Reasoning

arXiv: 2309.16436

Key Idea: LLM generates plans, Z3 SMT solver validates and produces counterexamples, counterexamples are fed back to the LLM for revision.

What Problem Does It Solve?

This is the oldest and simplest version of the idea you're using. In a classic blocksworld domain (stack blocks in a specific order), LLMs make plausible but incorrect plans. Jha et al. show that if you take the LLM's plan, translate it into SMT constraints, and ask Z3 to check it, Z3 can either confirm the plan is valid or produce a specific counterexample showing why it fails. That counterexample, phrased in natural language, is sent back to the LLM, which revises its plan. This loop repeats until the plan is verified correct or a maximum number of iterations is reached. They tested with GPT-4, GPT-3.5, and smaller models, showing the loop helps all of them.

Analogy: A student (LLM) writes a math proof. A teacher (Z3) doesn't just say "wrong" — they circle the exact step where the logic breaks and write "this doesn't follow because X." The student rewrites the proof. This continues until the proof is correct. Jha et al. proved this loop works. Your contribution is bringing it to a much harder domain (smart home scheduling with real physics) where it has never been tried.

1.5 Summary: How The Four Papers Relate

Paper	Role In Your Work	What You Take From It
SimuHome (ICLR 2026)	Target benchmark. The thing you improve on.	The simulator, the 600 QT4 episodes, the baseline numbers, the named failure mode (deferred feedback).
AGENT-C (arXiv 2025)	Nearest prior work. Your novelty check.	The concept of SMT-verified temporal tool-call constraints. You extend it to continuous physics-aware scheduling.
EvoPlan (arXiv 2026)	Architectural template. The loop you follow.	The propose-validate-repair pattern: LLM proposes, symbolic checker validates, feedback refines.
Jha et al. (arXiv 2023)	Foundational proof. The idea works at all.	The LLM + Z3 counterexample feedback loop, proven on blocksworld planning.

Table 3: The four papers and their roles in your research.

Part 2: The Research Gap You Are Addressing

2.1 The Failure in Numbers

SimuHome's results paint a clear picture. Here is what happens when GPT-4.1 (the best model tested) encounters infeasible QT4 episodes — scheduling requests that contain contradictory or impossible timing:

Task	GPT-4.1 Errors	Contradiction Blindness (CB)	Contradiction Mishandling (CM)
QT4-1 Infeasible (Future Scheduling)	45 errors total	40 (89%)	5 (11%)
QT4-2 Infeasible (Dependency Scheduling)	32 errors total	25 (78%)	6 (19%)
QT4-3 Infeasible (Concurrent Scheduling)	33 errors total	30 (91%)	1 (3%)

Table 4: GPT-4.1 error breakdown on infeasible QT4 episodes. CB = the model didn't even notice the contradiction. CM = noticed but handled it wrong.

The overwhelming failure mode is Contradiction Blindness: the model proceeds as if the request is perfectly valid. It doesn't "choose" to ignore the contradiction — it literally does not detect it. This is because temporal arithmetic ("the dishwasher finishes in 59 minutes, it's currently 5:01 PM, the user said 5:30 PM, does that work?") requires precise calculation, and LLMs are fundamentally unreliable at multi-step arithmetic, especially when interleaved with natural language reasoning.

2.2 The Root Cause: Deferred Feedback

The SimuHome authors identified the mechanism behind this failure in Section 6.2 of their paper. It's a system-design problem, not a model-intelligence problem:

When the agent calls `schedule_workflow` (the tool for scheduling future actions), it receives back only a bare acknowledgment: "workflow scheduled." The tool does NOT check whether the scheduled actions will actually be executable when their time comes. It does not verify that the timing is internally consistent. It does not flag contradictions between relative times ("10 minutes after the washer finishes") and absolute times ("at 5:30 PM"). It accepts everything.

Compare this with QT3 (explicit device commands), where 40% of successful episodes involved recovery after an initial tool-call error. In QT3, the tool immediately returns "error: device must be turned on before setting fan speed." The agent reads this, adjusts, and succeeds. Scheduling has no such mechanism — the agent flies blind.

Analogy: QT3 is like a compiler that immediately shows you syntax errors on the line where they occur — you fix them one by one and your code works. QT4 is like a compiler that says "compilation successful!" no matter what, and your program only crashes at runtime, hours later, with no stack trace. Your proposed contribution is building that missing compiler for smart home schedules.

2.3 Why Existing Work Doesn't Fix This

Three reasons existing approaches fall short:

A. Fine-tuning doesn't work. The SimuHome authors explicitly tested supervised fine-tuning on successful scheduling trajectories. It helped with detecting infeasible requests slightly but did not improve feasible scheduling success. The reason: each scheduling episode has different devices, different countdown times, different absolute times. The correct answer requires arithmetic that changes every episode — you can't memorize it from training examples.

B. AGENT-C handles the wrong kind of constraints. AGENT-C's temporal constraints are over discrete event orderings ("A must happen before B", "never call C twice"). Smart home scheduling requires reasoning about continuous, real-valued quantities: durations in minutes, countdown timers ticking down, environmental variables changing at rates determined by device settings. Z3 can handle real arithmetic — but nobody has built the encoding that maps Matter-protocol device semantics into SMT constraints over these continuous quantities. (Verify this by reading AGENT-C sections 3-4.)

C. EvoPlan and Jha et al. operate in different domains. Both prove that the LLM + solver loop works, but in robotics (spatial constraints) and blocksworld (discrete stack operations). Nobody has applied this pattern to smart home scheduling where the "physics" is thermal dynamics, humidity curves, and appliance cycle durations coupled through a shared room environment.

YOUR GAP IN ONE SENTENCE: No existing system combines SMT constraint verification with LLM-based smart home agents in a domain where device actions have continuous, time-varying effects on the environment. You are building that system and evaluating it on the first benchmark that actually tests this (SimuHome QT4).

Part 3: What You Are Building

3.1 High-Level Architecture

Your system sits between the LLM agent and SimuHome's `schedule_workflow` tool. It intercepts every scheduling attempt, verifies it against a formal constraint model, and either approves the schedule or returns a specific, actionable explanation of why it fails. The system has three components:

Component 1: Intent Parser (Small Language Model). A fine-tuned small model (Qwen3-4B with LoRA, or Phi-3-mini) that takes the natural-language user query plus the current device state (fetched via `get_device_structure`, `get_attribute`, `get_current_time`) and outputs a typed Intermediate Representation (IR). The IR is not the answer — it's a structured extraction of what the user wants:

IR Field	What It Captures	Example Values
Device	Which device(s)	washer_1, kitchen_light_1
Action	What to do	start, turn_off
TimeExpr	When (absolute / relative / dependency)	"in 10 min", "when washer finishes", "at 5:30 PM"
Duration	How long the device takes	59 min (from CountdownTime attribute)
Constraint	Implicit physical rules	"washer must be ON to start", "AC must be ON before setting temp"

Table 5: Fields of the typed Intermediate Representation.

Analogy: The Intent Parser is like a receptionist who listens to a patient's rambling description of symptoms and writes a structured intake form ("pain: left knee, onset: 3 days ago, severity: 7/10"). It doesn't diagnose anything — it just converts messy language into a structured form that the doctor (Z3) can work with.

Component 2: Constraint Encoder + Z3 SMT Solver. This is the core contribution. It takes the IR and encodes it as an SMT problem over real-valued variables. Specifically:

Time variables are Real-valued in Z3. "In 10 minutes" becomes $t_{\text{action}} = t_{\text{now}} + 10$. "When the dishwasher finishes" becomes $t_{\text{action}} = t_{\text{now}} + \text{countdown_dishwasher}$. "At 5:30 PM" becomes $t_{\text{action}} = 17.5$ (hours as decimal).

Consistency constraints are the rules Z3 checks. If the user says "10 minutes from now at 5:30 PM" and it's currently 5:25 PM, the system encodes: $t_{\text{now}} + 10 = 5:30 \text{ PM}$ AND $t_{\text{now}} = 5:25 \text{ PM}$. Z3 immediately finds this unsatisfiable ($5:35 \neq 5:30$) and returns the conflicting pair as an unsat core.

Duration constraints encode device operation times. "Schedule dryer to finish before 6 PM" with $t_{\text{now}} = 5:00 \text{ PM}$ and $\text{dryer_duration} = 90 \text{ min}$ becomes: $t_{\text{start}} + 90 \leq 360$ (6 PM in minutes from midnight) AND $t_{\text{start}} \geq 300$ (5 PM). Z3 finds this unsat (earliest finish is 6:30 PM) and reports the specific numbers.

Dependency constraints encode chains. "Turn off light when washer finishes, then start dryer 20 minutes later" becomes: $t_{\text{light_off}} = t_{\text{washer_done}}$ AND $t_{\text{dryer_start}} = t_{\text{light_off}} + 20$ AND $t_{\text{washer_done}} = t_{\text{now}} + \text{countdown_washer}$.

Physics constraints (optional, for bonus contribution) encode environmental effects. "Cool the room to 22C before guests arrive at 6 PM" with current temp = 28C and AC cooling rate = 0.5C/min becomes: $t_{\text{now}} + (28 - 22) / 0.5 \leq t_{\text{guests}}$. The cooling rate is derived from the Aggregator equations in SimuHome's appendix A.12 and can be linearized for Z3.

Analogy: The Constraint Encoder is like a legal contract reviewer. The Intent Parser hands it a deal sheet ("Party A delivers goods by March 1, Party B pays within 30 days of delivery, entire contract must close before February 15"). The reviewer doesn't need to understand the business — it just checks the dates: if delivery is March 1, payment is March 31, but the contract must close by Feb 15, that's impossible. Z3 is that reviewer, operating on times and durations instead of dates and deadlines.

Component 3: Feedback Verbalizer. When Z3 returns UNSAT (the schedule is impossible), it also returns an unsat core — the minimal set of constraints that conflict. The Verbalizer translates this back into natural language: "The washer finishes at 5:59 PM, but you asked for the dryer to be done by 5:30 PM. The dryer takes 90 minutes, so the earliest it can finish is 7:29 PM. Would you like to adjust the deadline?" This message is either returned to the user (for infeasible requests) or fed back to the LLM agent for plan revision (for feasible requests that the agent initially got wrong).

When Z3 returns SAT (the schedule is consistent), the verified schedule is automatically converted into the correct `schedule_workflow` API call and executed. The agent doesn't need to format the API call itself — the verified IR already contains all the information.

3.2 System Flow (Step by Step)

Step	Component	What Happens
1	User sends query	"Schedule the dryer to finish at the same time the washer does."
2	Base agent (ReAct) calls info tools	get_current_time -> 5:01 PM get_room_devices(utility_room) -> washer_1, dryer_1 get_attribute(washer_1, CountdownTime) -> 59 min get_attribute(dryer_1, ...) -> duration: 90 min
3	Intent Parser (fine-tuned SLM)	Extracts IR: devices: [washer_1, dryer_1] constraint: t_washer_done == t_dryer_done t_washer_done = now + 59 min = 6:00 PM dryer_duration = 90 min
4	Constraint Encoder -> Z3	Encodes: t_dryer_start + 90 = t_washer_done t_dryer_start >= t_now (can't start in the past) t_washer_done = t_now + 59 Z3 solves: t_dryer_start = 4:30 PM But t_now = 5:01 PM, so t_dryer_start < t_now -> UNSAT
5a	If SAT: Execute	Convert verified schedule to schedule_workflow API call and commit.
5b	If UNSAT: Verbalize	"The washer finishes at 6:00 PM, but the dryer takes 90 min. To finish together, the dryer needed to start at 4:30 PM, which is already past. The earliest the dryer can finish is 6:31 PM."
6	Return to user or revise	For infeasible: explain the contradiction. For feasible but wrong plan: feed the explanation back to the LLM to revise.

Table 6: End-to-end flow of the neuro-symbolic verification system.

3.3 Tech Stack

Component	Tool / Model	Details
Simulator	SimuHome	github.com/holi-lab/SimuHome CC BY-NC-ND 4.0 license. Python-based, runs locally.
Base Agent Framework	ReAct (same as SimuHome paper)	Ensures fair comparison with their baselines. Prompt template in their Appendix A.8.
Intent Parser (SLM)	Qwen3-4B + LoRA fine-tuning	Open-weight, fits on Colab T4/L4. Alternatives: Phi-3-mini (3.8B), Qwen3-1.7B (if compute-limited).
Training Framework	HuggingFace Transformers + PEFT	Standard LoRA fine-tuning. Dataset: synthetic IR pairs generated from SimuHome episodes.
SMT Solver	Z3 (Microsoft) via z3-solver Python package	pip install z3-solver. Mature, fast, well-documented. Supports real arithmetic natively.
Evaluation	SimuHome's built-in evaluation pipeline	Simulator-based for feasible; LLM-judge for infeasible. Direct comparison with Table 2.
Compute	1x GPU (Colab Pro / Kaggle P100/T4)	SLM fine-tuning: ~4-8 hours. Z3 solving: CPU-only, milliseconds. SimuHome runs: CPU-only.

Table 7: Complete tech stack. No hardware required. Everything runs in simulation.

HARDWARE NOTE: Z3 runs on CPU and solves constraints of the complexity you need (5-20 variables, 10-50 constraints per schedule) in under 100 milliseconds. The SLM runs on a single consumer GPU. SimuHome itself is a Python process with no GPU requirement. The entire pipeline can run on a Colab Pro notebook or a Kaggle kernel with a single GPU.

3.4 Training Data Generation (No Manual Labeling)

You need training data to fine-tune the Intent Parser (SLM) to convert natural-language queries into typed IRs. Here is how to generate it without any manual annotation:

Step 1: Use SimuHome's episode generation pipeline (Section 4.2 of the paper). Each episode already comes with: (a) the natural-language query, (b) the goal (target device states and times), (c) the required actions, and (d) the initial home state. The 600 episodes in the benchmark give you 200 QT4 episodes (50 per sub-type, feasible + infeasible).

Step 2: Write a script that converts each episode's goal + required actions into the IR format. This is deterministic: the goal says "washer off at 6:00 PM, kitchen light on at 6:00 PM," so the IR has `devices=[washer_1, kitchen_light_1]`, `times=[6:00 PM, 6:00 PM]`, `actions=[off, on]`. No LLM needed for this step.

Step 3: Augment by generating more episodes with varied initial states (SimuHome supports random seed-based initialization). Target 2000-5000 (query, IR) pairs for fine-tuning. This can be done programmatically in a few hours.

3.5 Planned Experiments and Ablations

The paper needs three things: (1) a main result showing improvement over baselines, (2) ablations proving the improvement comes from the symbolic layer, and (3) analysis of failure cases showing where the method still falls short.

Experiment	Setup	What It Shows
E1: Main Result	Run your full system (SLM + Z3 + Verbalizer) on all 200 QT4 episodes	Success rate on QT4-1/2/3 (feasible + infeasible). Compare against Table 2 baselines (GPT-4.1, Qwen3, etc.)
E2: Ablation — No Z3	Run SLM alone (no constraint checking) on same episodes	Shows the improvement is from the solver, not just from using a different model.
E3: Ablation — No SLM	Use GPT-4.1 as intent parser + Z3	Shows the SLM is sufficient (or not) vs. a frontier model for parsing.
E4: Ablation — No feedback loop	Z3 checks but doesn't feed back corrections; just blocks bad schedules	Shows the value of the repair loop vs. one-shot verification.
E5: Latency and cost	Measure wall-clock time and API cost per episode	Show your system is faster/cheaper than GPT-4.1 brute force.
E6: Error Analysis	Categorize remaining failures by type (using SimuHome's error taxonomy from Table 2)	Show which errors the symbolic layer fixes (TR, CB) and which it can't (DC, II).

Table 8: Planned experiments. E1 is the headline; E2-E4 are the ablations reviewers will demand.

TARGET NUMBERS: On QT4 feasible, you want to move from GPT-4.1's 34-50% to 65-80%+. On QT4 infeasible, you want Contradiction Blindness to drop from 75-91% of errors to near zero (since Z3 catches contradictions by construction). These are realistic targets because the temporal arithmetic errors are systematic, not random — a solver eliminates them entirely, and the remaining errors will be intent-parsing or device-selection mistakes that the symbolic layer is not designed to catch.

3.6 Execution Timeline (12 Weeks to Dec 15 Deadline)

Period	Tasks
Week 1-2	Read all 4 papers in full. Reproduce SimuHome baselines locally. Verify AGENT-C's constraint scope. Run SimuHome on Qwen3-32B to confirm you can reproduce Table 2 numbers.
Week 3-4	Design the IR schema. Write the Z3 constraint encoder for QT4-1 (future scheduling — the simplest sub-type). Test on 50 QT4-1 episodes manually (no SLM yet, hand-craft the IR).
Week 5-6	Extend constraint encoder to QT4-2 (dependency) and QT4-3 (concurrent). Build the Feedback Verbalizer. Generate synthetic training data for the Intent Parser.
Week 7-8	Fine-tune the SLM (Qwen3-4B + LoRA) on the synthetic IR dataset. Integrate all three components into the SimuHome ReAct loop. Run E1 (main result) on all 200 QT4 episodes.

Period	Tasks
Week 9-10	Run ablations E2-E6. Collect numbers. Debug any integration issues. Iterate on the IR schema if parsing accuracy is low.
Week 11-12	Write the paper. Format for IEEE IoT-J. Polish figures. Proofread. Submit by Dec 15.

Table 9: Week-by-week execution plan.

3.7 Risks and Mitigations

Risk	Severity	Mitigation
AGENT-C already covers continuous constraints	HIGH	Read their sections 3-4 immediately. If confirmed, pivot to Direction 2 (world-model + MCTS).
Someone publishes the same idea on SimuHome before you	MEDIUM	Check Google Scholar "cites SimuHome" weekly. Email the SimuHome authors to ask what extensions they've seen.
SLM intent parsing is too inaccurate	MEDIUM	Ablation E3 uses GPT-4.1 as parser instead. If that works and SLM doesn't, the contribution is the Z3 layer, not the SLM.
Z3 encoding of physics is too complex	LOW	Start with pure time/duration constraints (no physics). Physics is bonus. The paper is publishable without it.
SimuHome license (CC BY-NC-ND 4.0) blocks you	LOW	You're using it as a benchmark, not redistributing modified code. Research use is standard.

Table 10: Risk assessment.

3.8 One-Paragraph Summary of Your Contribution

You propose a neuro-symbolic verification layer for LLM-based smart home agents that addresses the deferred-feedback problem identified in SimuHome (ICLR 2026). A fine-tuned small language model parses natural-language scheduling requests into a typed intermediate representation, which is encoded as an SMT constraint problem over real-valued time, duration, and dependency variables. The Z3 solver either produces a provably consistent schedule (which is committed to the simulator) or returns a minimal unsat core that is verbalized into a specific, actionable explanation of why the schedule is impossible. This directly eliminates Contradiction Blindness — the dominant failure mode on SimuHome's hardest benchmark category — by shifting temporal consistency checking from the LLM's unreliable internal arithmetic to a sound, complete decision procedure. The system requires no hardware (all experiments run in SimuHome's simulator on a single GPU), combines proven components (SMT verification from AGENT-C's paradigm, propose-validate-repair from EvoPlan's architecture) in a domain where they have never been applied, and targets the IEEE IoT-J special issue on AI Agent Enabled Small-Large Model Collaboration for IoT.